

Apresentação do Projeto: DocuDigit - Classificador de Dígitos Manuscritos com Análise Preditiva

Apresentador: Marco Aurélio Schlemper

Contexto: Mini-Projeto Avaliativo - Módulo 2 (Ciência de Dados e Machine Learning)

1. Introdução e Objetivos do Sistema

O **DocuDigit** foi desenvolvido para resolver o desafio de processamento de dados não estruturados e classificação multiclasse. O objetivo principal do sistema é construir, treinar, validar, comparar e estressar um pipeline preditivo ponta a ponta utilizando o dataset clássico **MNIST** ([mnist_784](#)), que contém 70.000 imagens de dígitos manuscritos de 0 a 9 em escala de cinza (28 x 28 pixels).

2. Metodologia e Organização (Git e GitHub)

O versionamento do código seguiu rigorosas boas práticas de engenharia de software:

- Utilização de uma branch principal ([main](#)) para a entrega final e estável.
- Criação de uma branch intermediária ([develop](#)) para centralizar os merges.
- Uso de *feature branches* dedicadas para cada fase do projeto (como EDA, modelagem e testes OOD).
- Estruturação do repositório contendo caminhos estritamente relativos, arquivo [requirements.txt](#) com as dependências fixadas e o conjunto de imagens manuscritas digitalizadas ([CARACTER0.png](#) a [CARACTER9.png](#)) posicionadas na raiz.

3. Desenvolvimento Passo a Passo e Resultados

Fase 1: Carregamento e Análise Exploratória de Dados (EDA)

- **Procedimento:** O dataset [mnist_784](#) foi importado via [scikit-learn](#) ([fetch_openml](#)). A estrutura matricial resultante possui dimensões de 70.000 amostras por 784 colunas (correspondendo ao achatamento das matrizes bidimensionais de 28 x 28 pixels), acompanhadas pelo vetor de rótulos y .
- **Resultados e Visualização:** A distribuição das classes provou estar perfeitamente balanceada. Através da biblioteca [matplotlib](#), gerou-se uma grade visual de 2 x 5 exibindo exemplos reais de cada dígito. Constatou-se tecnicamente que cada pixel armazena intensidades de cinza variando de 0 (preto) a 255 (branco).

Fase 2: Pipeline de Pré-processamento e Divisão dos Dados

- **Procedimento:** Aplicou-se a divisão estratificada dos dados ([stratify=y](#)) em conjuntos de treino, validação e teste para salvaguardar a integridade estatística e mitigar o risco de *data leakage*. Em seguida, realizou-se a normalização dos pixels dividindo todos os valores por 255.0, redimensionando a escala para o intervalo contínuo de $[0.0, 1.0]$.

- **Justificativa:** A normalização foi essencial para garantir a convergência matemática de modelos lineares e distâncias métricas, evitando que features com valores brutos maiores distorcessem os cálculos de otimização.

Fase 3: Implementação e Treinamento de Três Modelos Distintos

Três abordagens algorítmicas diferentes foram treinadas, cada qual com o ajuste de pelo menos dois hiperparâmetros fundamentais:

1. **Random Forest:** Ajustados `n_estimators` (número de árvores de decisão para robustez do ensemble) e `max_depth` (limitação de profundidade para controle de *overfitting*).
2. **K-Nearest Neighbors (KNN):** Ajustados `n_neighbors` (quantidade de vizinhos espaciais avaliados) e `weights='distance'` (atribuindo maior peso aos vizinhos mais próximos).
3. **Perceptron Multicamadas (MLP / Rede Neural):** Ajustados o número de neurônios nas camadas ocultas e a taxa de aprendizado inicial (*learning rate*) para capturar padrões não lineares complexos.

Fase 4: Avaliação Comparativa de Desempenho

- **Procedimento:** Os três modelos foram avaliados sobre o conjunto de teste independente. Gerou-se o relatório de classificação completo e as Matrizes de Confusão completas de 10×10 com mapas de calor.
- **Resultados Consolidados:**
 - A tabela comparativa evidenciou métricas robustas de Acurácia Global, Precisão Média Ponderada, Recall e F1-Score para os três algoritmos.
 - **Análise Crítica:** Observou-se um *trade-off* claro entre acurácia e custo computacional. Modelos baseados em redes neurais e ensembles entregaram alta performance preditiva em troca de maior tempo de treinamento, enquanto o KNN apresentou tempo de inferência elevado. As maiores taxas de confusão concentraram-se entre pares de dígitos geometricamente semelhantes, como o **4 e o 9** ou o **3 e o 5**.

Fase 5: Testes de Estresse, OOD e Inferência Real

- **Fase 5.1 e 5.2 (Class Masking e Generalização Extrema - OOD):** Oculpou-se propositalmente duas classes do treinamento (dígitos **4 e 7**). Ao submeter o modelo a um conjunto de teste contendo exclusivamente essas classes ocultas, analisou-se o fenômeno da **falsa certeza (overconfidence)**: o classificador, forçado a rotular dados que nunca viu, tendeu a mapeá-los incorretamente para as classes conhecidas atribuindo-lhes níveis de confiança estatisticamente elevados.
- **Fase 5.3 (Inferência com Imagens Próprias):** Utilizando as imagens digitalizadas pelo próprio autor (`CHARACTER0.png` a `CHARACTER9.png`), construiu-se um pipeline de visão computacional em Python (via Pillow) contendo conversão para escala de cinza, inversão de cores (fundo escuro e traço claro, equiparando-se ao padrão MNIST), redimensionamento para 28×28 pixels com centralização de massa e normalização. O modelo de melhor desempenho realizou com sucesso a predição das imagens reais, cujos resultados de probabilidade foram plotados graficamente ao lado de cada caractere.

4. Conclusão e Trabalhos Futuros

O **DocuDigit** cumpriu integralmente os requisitos propostos na ementa, evidenciando a eficácia na construção de pipelines de Machine Learning estruturados, desde a engenharia de dados até testes rigorosos de robustez em ambientes adversos. Como melhoria futura, aponta-se a migração para arquiteturas de **Redes Neurais Convolucionais (CNNs)**, capazes de extrair automaticamente características espaciais bidimensionais sem a necessidade de achatamento vetorial.